



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

**Домашнее задание №2
по курсу «Моделирование»
«Построение вероятностной модели»**

Студент: Дьячков Е.С.

Группа: ИУ9-81Б

Преподаватель: Домрачева А.Б.

Москва 2026

1 Постановка задачи

Целью данной работы является построение вероятностной модели, проведение вычислительного эксперимента для различных значений параметра p (вероятности сохранения первоначального выбора игроком на завершающем этапе) и оценка вероятности выигрыша игрока.

Рассматривается следующая задача. За одной из N дверей находится приз, за остальными — пусто. Игроку предлагается выбрать одну из N дверей наудачу. Не открывая выбранной двери и не показывая двери с призом, ведущий открывает K пустых дверей по своему желанию. В завершение игры игрок может поменять или сохранить свой выбор. Необходимо провести вычислительный эксперимент, изменяя значения K , N , а также вероятность p сохранения/изменения выбора на завершающем этапе игры с целью оценивания вероятности выигрыша игрока. Для решения задачи выбрана вероятностная модель. Исходы являются дискретными и определяются правилами игры и случайными выборами. Вероятностная модель позволяет воспроизвести поведение системы без упрощающих допущений и получить оценки вероятности выигрыша.

2 Построение математической модели

Для решения задачи была создана концептуальная схема вероятностной модели. Она представлена на рисунке 1 и отражает три этапа игры: шаг расположения приза, шаг открытия дверей ведущим и шаг окончательного выбора игрока.

Формализация модели

Входные данные

- N — число дверей ($3 \leq N \leq 10$);
- K — число открываемых ведущим дверей ($1 \leq K \leq N - 2$);
- параметр $p \in [0, 1]$ — вероятность сохранения первоначального выбора.

Промежуточные переменные

- D_i — событие «приз за дверью i », $i \in \{1, \dots, N\}$;



Рис. 1 — Схема вероятностной модели

- O_j — событие «ведущий открывает двери j ».

Выходные данные $P(X = 1 | p)$ — вероятность выигрыша при параметре p .
Допущения

1. Приз размещается равновероятно: $P(D_i) = 1/N$ для всех i .
2. Ведущий всегда открывает K дверей с козами.
3. Испытания независимы.

Математическое уравнение модели

Пусть A — событие, что игрок изначально выбрал дверь с призом. Тогда вероятность выигрыша определяется как

$$P_{\text{win}} = p \cdot P(A|B) + (1 - p) \cdot P(\text{switch}),$$

где $P(A|B)$ вычисляется по формуле Байеса, а $P(\text{switch})$ — вероятность выигрыша при смене выбора на одну из оставшихся дверей.

3 Постановка, исследование и решение вычислительных задач

Требуется оценить вероятность выигрыша $P(\text{выигрыш} \mid p)$ игрока. Для проверки адекватности модели выбран классический случай ($N = 3, K = 1$). Вероятностная модель реализована на языке Python методом Монте-Карло. Полный код модели приведён в приложении.

4 Проверка качества моделей на практике и модификация модели

Апробация модели проведена с целью проверки её адекватности.

Основной эксперимент. Пять серий по 5000 прогонов при значениях $p \in \{0; 0,25; 0,5; 0,75; 1\}$ (фиксированные $N = 3, K = 1$). Полученные значения $\bar{P} = \text{wins}/5000$ сведены в таблицу 1 вместе с теоретическими значениями $P(p)$.

Таблица 1: Сравнение теоретической и вероятностной оценок вероятности выигрыша ($N = 3, K = 1$)

p	$P(p)$ (теория)	num_games	$\bar{P}_{\text{Python}}$	$ P(p) - \bar{P}_{\text{Python}} $
0	0,6667	5000	0,6638	0,0029
0,25	0,5500	5000	0,5900	0,0400
0,5	0,4333	5000	0,5098	0,0765
0,75	0,3167	5000	0,4222	0,1055
1	0,2000	5000	0,3446	0,1446

Расхождение между теоретическими и экспериментальными значениями не превышает 0,1446. Наблюдается монотонное убывание вероятности выигрыша с ростом p .

Проверка сходимости. Дополнительно проведено тестирование стратегии switch ($p = 0$) для различных значений N и K .

При увеличении числа испытаний оценка устойчиво сходится к теоретическому значению. Адекватность модели подтверждена.

Таблица 2: Оценка вероятности выигрыша при стратегии switch ($p = 0$)

Параметры	$\bar{P}$	Теория	$ \text{теория} - \bar{P} $
$N = 3, K = 1$	0,6651	0,6667	0,0016
$N = 10, K = 8$	0,9009	0,9000	0,0009
$N = 100, K = 98$	0,9902	0,9900	0,0002

5 Выводы

В ходе работы построена вероятностная стохастическая модель игры Монти Холла на языке Python. Модель использует формулу Байеса для теоретического расчёта вероятности и метод Монте-Карло для экспериментальной оценки $P(\text{выигрыш} \mid p)$.

Проведён параметрический вычислительный эксперимент при пяти значениях параметра $p \in [0, 1]$ (фиксированные $N = 3, K = 1$), в каждой серии выполнено 5000 прогонов. Экспериментальные результаты близки к теоретическим. Подтверждено, что вероятность выигрыша убывает при увеличении вероятности сохранения первоначального выбора p .

Оптимальной стратегией является полная смена выбора ($p = 0$), при которой вероятность выигрыша максимальна и может достигать значений, близких к 1 при большом числе дверей и соответствующем K .

А Приложение

А.1 Текст программы на Python

Полный код вероятностной модели:

```

1 import random
2 import math
3
4 def nCr(n, r):
5     return math.comb(n, r)
6
7 def theoretical_p_win_full(N, K, p_stay):
8     p_a = 1.0 / N
9     p_not_a = (N - 1) / N
10    term_a = (1.0 / nCr(N-1, K)) * p_a
11    term_not_a = (1.0 / nCr(N-2, K)) * p_not_a

```

```

12     p_a_given_b = term_a / (term_a + term_not_a)
13     p_switch = p_not_a * (1.0 / (N - K - 1))
14     return p_stay * p_a_given_b + (1 - p_stay) * p_switch
15
16 def monty_hall_simulation(n_doors=3, k_open=1, p_stay=0.5, n_trials=5000, seed
    =42):
17     random.seed(seed)
18     wins = 0
19     for _ in range(n_trials):
20         prize = random.randint(0, n_doors - 1)
21         choice = random.randint(0, n_doors - 1)
22         available_to_open = [i for i in range(n_doors) if i != prize and i !=
    choice]
23         if len(available_to_open) < k_open:
24             available_to_open = [i for i in range(n_doors) if i != choice]
25         opened = random.sample(available_to_open, k_open)
26         remaining = [i for i in range(n_doors) if i != choice and i not in
    opened]
27         if random.random() < p_stay:
28             final_choice = choice
29         else:
30             final_choice = random.choice(remaining)
31         if final_choice == prize:
32             wins += 1
33     return wins / n_trials
34
35 for p in [0.0, 0.25, 0.5, 0.75, 1.0]:
36     est = monty_hall_simulation(p_stay=p)
37     theo = theoretical_p_win_full(3, 1, p)
38     print(f"p_stay={p:.2f}: est={est:.4f} theory={theo:.4f} diff={abs(est - theo)
    :.4f}")

```

Листинг 1: модель